

```
$ python src/check_url.py "http://paypal.com.verify-login.ru"
```

```
Verdict: PHISHING          p=0.705
```

The Phishing Detector

Field Guide

How a 107,355-URL machine learning system was built, why every design choice was made, and how to explain all of it - code, flags, metrics, ethics - in your own words.

107,355

real URLs trained on

0.888

RandomForest F1 - honest, not fake 1.0

6 d.p.

Python vs browser prediction parity

The Big Picture

The project answers one question: **given only the text of a URL, is it a phishing link?** No network calls, no page content, no JavaScript - just the string. That constraint is deliberate: it makes the detector fast, private, and safe to demo (you never have to visit a live phishing page).

PIPELINE

```

URL string
|
v
features.py      extract_features(url) -> 18 numbers
|
+--> train.py    learn patterns from 107,355 labeled URLs
|                saves models/RandomForest.pkl (the brain)
|                models/LogisticRegression.pkl (the simple brain)
|
+--> check_url.py CLI: load pipeline -> extract -> predict -> explain
|
+--> workspace/  the same 18 features + exported LR model in your browser

```

One extractor, three consumers

The single most important architectural decision: **one file defines the features, and everyone else imports it.** Training, the CLI, and the web app all compute the same 18 numbers from a URL. If training and inference disagreed even slightly, the model would be judging the world by rules it was never taught. We call this train/inference parity, and we test it to 6 decimal places.

The cast of files

File	Role in one line
features.py	The 18 feature functions - the vocabulary of the model
features_test.py	81 unit tests guarding each feature's behavior
fetch_data.py	Builds the dataset: downloads, dedupes, caps per host
train.py	Trains both models inside sklearn Pipelines, saves .pkl + reports
evaluate.py	Honest test-set scores, confusion matrix, ROC curve
check_url.py	The CLI: check any URL, explain the verdict
export_model.py	Optional: exports LR to JSON for the web demo (quarantined)

IF YOU REMEMBER ONE THING

A machine learning model is a function from features to a prediction. 90% of this project's bugs were feature- or data-bugs, not model bugs. The model only knows what the numbers tell it.

The 18 Features - Teaching a Machine to Be Suspicious

A URL is just text, so we convert it into 18 numbers describing its shape. Think of them as **suspicion instincts**: things a trained human notices in half a second without opening the link. The model's job is to learn which instincts matter and how much.

The full vocabulary

#	Feature	What it measures	Why it helps
1	url_length	Total characters	Phishing URLs are often long (padding to look real)
2	hostname_length	Domain length	Deceptive domains are wordy
3	path_length	Length after the domain	Deep fake login paths (verified by our data fix)
4	num_dots	Dots in URL	More labels = more places to hide the real domain
5	num_hyphens	Hyphens	secure-paypal-login.com energy
6	num_underscores	Underscores	Rare in honest URLs
7	num_slashes	Slashes	Deep nesting, often fake structure
8	num_digits	Digit count	Random junk domains love digits
9	num_special_chars	@, %, =, &, ?...	Obfuscation machinery
10	has_at_symbol	Is @ present	Classic trick: browser trusts what is BEFORE the @
11	has_ip_address	Host is numbers	No domain = no reputation = red flag
12	num_subdomains	Labels before the TLD	paypal.com.verify-login.ru has 3
13	uses_https	Scheme is https	Weak signal: phishers use https too now
14	has_https_token_in_path	The word 'https' hiding in the path	Faking security in text
15	is_shortened_url	bit.ly, tinyurl...	Hides the true destination
16	url_entropy	Shannon entropy	Random-looking strings score high
17	tld_in_subdomain	A TLD name posing as a subdomain	The homograph detector - see below
18	digit_letter_ratio	Digits / letters	More junk, more digits

Deep dive 1: the @ trick

In `https://google.com@evil.ru/login` the browser connects to **evil.ru** - everything before @ is decoration. Humans read left to right and get fooled; the feature `has_at_symbol` catches it instantly. One boolean, enormous historical signal.

Deep dive 2: the bug that ate bbc.co.uk

Feature 17, `tld_in_subdomain`, was built to catch look-alike domains like **paypal.com.verify-login.ru** - the word `.com` hiding in the middle of the domain. The first version had an embarrassing bug:

FEATURES.PY (ABRIDGED)

```
# BEFORE - the naive version
parts = hostname.split('.')
if any(part in COMMON_TLDS for part in parts[:-1]):
    return 1          # fires on bbc.co.uk ('co' looks like a TLD!)

# AFTER - with the ccTLD exemption
for i, part in enumerate(parts[:-1]):
    if part not in common_tlds:
        continue
    # 'co' in bbc.co.uk sits right before a 2-letter ccTLD:
    # that is a legitimate multi-part suffix, not a trick
    if i == len(parts) - 2 and len(part) == 2:
        continue
return 1          # paypal.com.verify-login.ru still fires
```

WHY THIS BUG WAS DANGEROUS

4.1% of legitimate URLs (bbc.co.uk, site.com.au style) were being flagged as suspicious. The false signal drowned the true one - the homograph paypal.com.verify-login.ru briefly scored as LEGITIMATE. The lesson: a feature bug is a model bug. We now test both bbc.co.uk (must NOT fire) and paypal.com.verify-login.ru (must fire) in features_test.py - 81 tests total.

Deep dive 3: shortened URLs

is_shortened_url checks the host against a list of shorteners (bit.ly, tinyurl.com, t.co, goo.gl, is.gd, cutt.ly and friends). Shorteners are not evil - but they are the standard cloak for phishing in messages, and the model learned to treat them as a strong signal (our smoke test: bit.ly link scores $p=1.00$).

The Data - Three Stories, Three Traps

The final dataset: **107,355 real URLs** - 48,338 phishing + 59,017 legitimate. But the road here taught more than the destination. Each source below fixed a real modeling disaster.

The sources

Source	Class	What it gives
PhishTank verified feed (online-valid.csv)	phishing	48k community-verified live phishing URLs
Tranco top-10k list	legitimate	The 10,000 most-visited domains on Earth
ealvaradob/phishing-dataset benign subset	legitimate	~59k real benign deep links WITH paths
UCI Phishing Websites (id=327)	benchmark only	11k rows, 30 pre-computed features - NOT trained on

Why not UCI for training? Its 30 features are pre-computed and anonymized - there are no raw URLs, so our feature extractor cannot run on it. Worse, several features (page_rank, web_traffic, google_index, dnsrecord, age_of_domain) need live network lookups, breaking the offline promise. It stays cached as a benchmark reference only.

Story 1: the perfect score that was a lie

The first dataset had 244 URLs - and the model scored $F1 = 1.000$. Perfect. Too perfect. The phishing URLs were mostly .ru and .xyz, the legitimate ones .com - the model could separate the classes **by top-level domain alone**. It was not learning phishing; it was learning our sampling habits. Rule of thumb: a perfect score on a small, hand-made dataset is a bug report, not a result.

Story 2: any path = phishing (dataset bias)

Tranco only publishes bare homepages - google.com, never google.com/mail/u/0. PhishTank URLs usually have deep paths. So the model learned: **has a path = phishing**. It flagged github.com/python/cpython at 0.95. The fix was to add real benign URLs with paths, so both classes contain deep links:

DATASET AUDIT

```
# the bias, measured on the first dataset
# legit URLs containing a path : 0% (Tranco = homepages only)
# phishing URLs with a path : 61% (PhishTank = deep links)
# after adding ealvaradob benign URLs
# legit with paths : 77% phishing with paths : 61% -> no shortcut left
```

Story 3: the docs.google.com memorization

PhishTank contained 5,700+ URLs all hosted on docs.google.com (abusing Google Forms for phishing). A random forest happily memorizes host-level patterns: '3 dots + short host = bad'. That inflates test scores but fails in the real world. Fix: **host capping**.

FETCH_DATA.PY (ABRIDGED)

```
def _cap_per_host(df, max_per_host):
    """Keep at most max_per_host URLs per hostname."""
    return (df.groupby("host", group_keys=False)
            .apply(lambda g: g.head(max_per_host)))

# called for BOTH classes - fairness in capping:
phishing_df = _cap_per_host(phishing_df, max_per_host) # max 10/host
legit_df    = _cap_per_host(legit_df,    max_per_host)
```

Hygiene: deduplication

FETCH_DATA.PY (ABRIDGED)

```
# cross-class dedupe: if a URL is labeled BOTH ways, keep the benign one
legit_urls = set(legit_df["url"])
overlap = phishing_df["url"].isin(legit_urls).sum()
phishing_df = phishing_df[~phishing_df["url"].isin(legit_urls)]

# plus exact dedupe per class; final shuffle with a fixed seed
combined = pd.concat([...]).sample(frac=1, random_state=42)
```

THE SLEUTH'S SUMMARY

Bias lives in how data is collected, not in the model. Tranco-only taught 'paths are evil'; PhishTank-heavy taught 'docs.google.com is evil'. Every fix made metrics WORSE (F1 0.9255 -> 0.888) and the system BETTER - because the dropped points were lies the data told.

The Pipeline Fix - One Scaler to Rule the Right Model

The bug

Version 1 fit ONE StandardScaler and fed scaled data to RandomForest AND LogisticRegression, then saved scaler.pkl separately for inference. Two problems: trees do not need scaling at all, and a loose scaler file is a **leakage and mismatch hazard** - any drift between training-time scaling and inference-time scaling silently corrupts predictions.

The fix: preprocessing travels inside the model

TRAIN.PY (ABRIDGED)

```
lr_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(class_weight="balanced",
                              max_iter=1000, random_state=42)),
])

rf_pipeline = Pipeline([
    ("clf", RandomForestClassifier(n_estimators=200,
                                  class_weight="balanced",
                                  random_state=42, n_jobs=-1)),
]) # no scaler - trees are scale-invariant

joblib.dump(rf_pipeline, "models/RandomForest.pkl") # ONE file each
```

Now the .pkl is **self-contained**: load it, hand it raw numbers, get a prediction. check_url.py shrank to three lines of inference: joblib.load, predict, predict_proba. And cross-validation becomes automatically honest: every CV fold refits the WHOLE pipeline, so the scaler is computed from that fold's training data only - **zero leakage**, with no extra code.

Question	Answer to give in the viva
Why does LR need scaling?	Its loss weighs features by magnitude; unscaled big-range features dominate
Why doesn't RF need it?	Trees split by thresholds per feature - scale never matters
What is leakage?	Test-set information sneaking into training (e.g. scaler fit on all data)
How do we avoid it here?	Pipeline + cross_val_score: every fold refits scaler + model from scratch
What is class_weight=balanced?	Re-weights classes inversely to frequency so minority errors hurt more

Evaluation - Reading the Numbers Like an Adult

The headline results (test set = 21,471 URLs, stratified 80/20)

Model	Precision	Recall	F1	ROC-AUC	CV F1 (5-fold)
RandomForest	0.8885	0.8872	0.8878	0.9545	0.8890 +/- 0.0023
LogisticRegression	0.8124	0.7963	0.8043	0.8850	0.8015 +/- 0.0033

What each metric actually says

- **Precision 0.8885** - when the model shouts PHISHING, it is right 88.9% of the time. Protects legitimate sites from being blocked.
- **Recall 0.8872** - of all real phishing, it catches 88.7%. The rest sail past; each miss is a potential victim.
- **F1 0.8878** - the harmonic mean; punishes imbalance between precision and recall. Our single headline number.
- **ROC-AUC 0.9545** - rank a random phish above a random legit URL 95.5% of the time. Measures separability across ALL thresholds.
- **CV F1 +/- std** - five retrainings on different splits land within 0.0023 of each other: the score is stable, not a lucky draw.

The confusion matrix, decoded (RandomForest)

	Predicted LEGIT	Predicted PHISH
Actually LEGIT	TN = 10,727 (correct pass)	FP = 1,076 (false alarm)
Actually PHISH	FN = 1,091 (the misses)	TP = 8,577 (caught)

Read it as a story: of 9,803 real phishing attacks in the test set, the model catches 8,577 and misses 1,091; of 11,803 legitimate URLs it wrongly accuses 1,076. That FP/FN tension is the deployment tradeoff - and it is adjustable live with threshold (Chapter 6).

WHY 0.888 IS THE HONEST NUMBER

The brief expects F1 roughly in 0.85-0.97 on real data. Our 0.888 sits there because real phishing is genuinely hard: top phishing kits perfectly imitate real login pages, so pure URL shape has a ceiling. A perfect 1.0 on tiny data was the red flag; a stable 0.89 with tight CV variance is a result you can defend.

The CLI - check_url.py, Flag by Flag

The one-liner anatomy

USAGE

```
$ python src/check_url.py <url> [--model NAME] [--threshold 0.5]
  [--json] [--list-models]
```

Flag	What it does inside the program
<url> (positional)	The URL to judge. If the scheme is missing, http:// is assumed and a note is printed. No network call is EVER made - the string is only parsed.
--model NAME	Which .pkl brain to load: RandomForest (default, 200 trees, ~53 MB) or LogisticRegression (2 KB, ~10 s faster startup, slightly lower accuracy).
--threshold X	Decision boundary, default 0.5. Lower = paranoid (more phishing calls, more false alarms); higher = only near-certain calls. Deployments tune this to taste.
--json	Machine-readable output: verdict, probability, all 18 features, and the full contribution list. Perfect for piping into other tools or screenshots.
--list-models	Lists every .pkl in models/ - a sanity check that training actually saved something.

How a verdict is produced

CHECK_URL.PY (THE WHOLE INFERENCE PATH)

```
pipeline = joblib.load("models/RandomForest.pkl") # scaler included
feats = extract_features(url) # 18 numbers
X = pd.DataFrame([feats], columns=get_feature_names())
pred = pipeline.predict(X)[0] # 0 or 1
prob = pipeline.predict_proba(X)[0, 1] # p(phishing)
pred = 1 if prob >= args.threshold else 0 # user's boundary
```

The explain button - occlusion, explained

Global feature importances say what the model USUALLY cares about. They cannot explain a single verdict. So check_url.py runs a counterfactual experiment per URL: swap one feature to a benign-typical value, re-predict, and measure how much the probability moved:

CHECK_URL.PY (ABRIDGED)

```
for idx, name in enumerate(feature_names):
    ref = _BENIGN_REFERENCE[name] # e.g. has_at_symbol -> 0.0
    Xc = X.copy()
    Xc[0, idx] = float(ref) # 'occlude' this feature
    p = pipeline.predict_proba(Xc)[0, 1]
    contribution = base_prob - p # + pushed toward PHISHING
```

ANNOTATED EXAMPLE OUTPUT

```
$ python src/check_url.py http://paypal.com.verify-login.ru/user/login
Verdict: PHISHING Phishing probability: 0.7050

Top feature contributions (toward PHISHING + / toward LEGITIMATE -):
- num_subdomains = 3 -> +0.31 (the 'com' hiding in the middle)
- tld_in_subdomain = 1 -> +0.22 (the homograph rule fired)
- num_dots = 4 -> +0.09
- uses_https = 0 -> +0.04
Benign baseline: 0.0340 - what the model would say if EVERY feature
were typical of a legitimate URL.
```

The import shim (a real-world papercut)

The modules use flat imports (from features import ...), which works when run as scripts but broke `python -m src.check_url`. The fix - try package imports first, fall back to flat - is a reusable pattern:

DUAL-MODE IMPORT

```
try:    # package mode:  python -m src.check_url
    from .features import extract_features, get_feature_names
except ImportError:  # script mode:  python src/check_url.py
    from features import extract_features, get_feature_names
```

DEMO RECIPE FOR THE VIDEO

Run these in order: www.google.com (legit), github.com/python/cpython (legit deep link), paypal.com.verify-login.ru (phish), bit.ly link (phish). Then rerun one with `--threshold 0.3` to show the tradeoff, and finally `--json` to show the machine output. Sixty seconds, complete story.

The Web App - Same Brain, Different Body

A React + TypeScript demo (workspace/) runs the LogisticRegression model **in your browser, offline**. The LR pipeline is just arithmetic - $z = \text{bias} + \sum(w_i * (x_i - \text{mean}_i) / \text{std}_i)$, $p = \text{sigmoid}(z)$ - so 2 KB of JSON reproduces it exactly. The 200-tree forest cannot be exported losslessly; the browser app runs LR and cites the Python CLI for RF.

Proven parity

PARITY TEST

URL	Python LR	Browser LR	
paypal.com.verify-login.ru/...	0.821943	0.821943	
bit.ly/3xY2zAb	0.891398	0.891398	
www.google.com	0.0004xx	0.0004xx	(all 10 test URLs match to 6 d.p.)

The no-drift chain

Early on, the export had stale, hand-copied metrics - a maintenance trap. Now numbers flow one way only, with no human in the loop:

SYNC ARCHITECTURE

```
train.py finishes
-> writes reports/training_report.json (the single source of truth)
-> calls export_model.py automatically (best-effort, never fatal)
    reads training_report.json LIVE - nothing is baked in
    if the export is older than the .pkl files: loud STALE warning
-> writes workspace/src/model/lrModel.json (model + metrics)
-> web app derives ALL displayed numbers from that JSON
```

Scope note for the report: the deliverable is the model + CLI; the web app is a clearly quarantined optional extra. Deleting export_model.py and workspace/ removes it with zero impact on the Python project.

The Defensive-Engineering Playbook

Six traps this project fell into and climbed out of. Each is a story you can tell in the viva - graders love honest failure analysis more than fake perfection.

Trap	Symptom	Fix	Lesson
Too-separable toy data	F1 = 1.000 on 244 URLs	Real sources: PhishTank + Tranco + ealvaradob	Perfect scores on small data = sampling artifact
Shared scaler file	One scaler.pkl served two models	Pipeline per model; scaler lives inside .pkl	Preprocessing belongs to the model
'Paths are evil' bias	github deep links flagged 0.95	Add benign URLs WITH paths (77% now)	Balance shape, not just labels
Host memorization	5.7k docs.google.com URLs; host-pattern overfit	Cap 10 URLs per host, both classes	Cap sources, not classes
Feature bug (ccTLD)	bbc.co.uk flagged; homographs missed	ccTLD exemption + regression tests	A feature bug IS a model bug
Export drift	Stale metrics baked into JSON	Auto-export after training; live report reads; stale alarm	Single source of truth, zero hand-copied numbers

THE META-LESSON

Every fix made the metrics worse and the system better. That inversion - dropping F1 from 0.9255 to 0.888 while deleting lies from the data - is the difference between optimizing a number and engineering a system.

Ethics, Limits, and Honest Failure Modes

Bias - inherited from the sources

- PhishTank skews toward campaign phishing (hosted kits, Google Forms abuse); Tranco/ealvaradob skew toward popular sites. Long-tail legitimate domains and novel phishing kits are underrepresented.
- Language and region bias: the sources skew toward English-language and Western URLs; TLD-heavy regions may see different error rates.

Misuse - how a detector becomes a weapon

An attacker can use the CLI as a free oracle to mutate a phishing URL until it scores LEGITIMATE - adversarial optimization. Mitigations in practice: rate-limit public scoring endpoints, combine lexical scores with content/reputation systems, and never rely on a single static model.

False positives have victims too

A flagged legitimate URL can be blocked, costing businesses trust and money. That is why precision is reported as prominently as recall, and why threshold exists: a bank's filter would set a higher bar; a mail provider a lower one.

Hard limits of lexical-only detection

- Homoglyph IDN attacks (paypal.com with a Cyrillic 'a') look almost normal as ASCII features.
- A clean-looking domain serving evil content passes - we never see the page.
- Novel TLD abuse (.zip, .mov style) can drift the feature distribution over time - models need periodic retraining on fresh feeds.

Mitigations, summarized

- Diverse sources + host capping + dedupe (done) - reduces the loudest biases.
- Documented limits (this chapter) - no overclaiming.
- Human in the loop - the tool ASSISTS judgment; it should not be the sole decision-maker.
- Scheduled retraining from live feeds - the pipeline (fetch -> train -> evaluate) is one command each.

Viva Cram Sheet - Twelve Questions You Will Be Asked

Q1. Why two models?

RandomForest for accuracy (captures feature interactions, F1 0.888); LogisticRegression for transparency and portability (2 KB, exports to the browser). Comparing a complex and a simple model also shows whether complexity is earning its keep - it is (0.888 vs 0.804).

Q2. Why not deep learning / neural networks?

Tabular data of 18 engineered features is classic tree territory; a neural net would need more data, more compute, and would lose the per-feature explanations. Right tool for the right job.

Q3. Why is F1 not 1.0, and why is that GOOD?

Because the data is real and the classes genuinely overlap - phishing kits imitate legitimate sites. The old 1.0 came from 244 URLs separable by TLD. Real, honest, defensible.

Q4. How do you prevent data leakage?

Three layers: stratified split BEFORE any fitting; scaler inside the Pipeline so CV refits it per fold; cross-class URL deduplication so the same URL cannot sit in train and test with different labels.

Q5. What exactly is a sklearn Pipeline?

A chained object of preprocessing steps + estimator that fits/predicts as one unit. It guarantees inference applies the exact same transforms as training, and makes the saved .pkl self-contained.

Q6. Why cap 10 URLs per host?

PhishTank had 5,700+ URLs on docs.google.com. Without capping the forest memorizes host patterns instead of transferable lexical signals - high test scores, poor real-world generalization.

Q7. Why is Tranco not enough for the legit class?

Tranco is bare homepages only - zero paths - while 61% of phishing has paths. The model learned 'path = phishing'. The ealvaradob benign set (77% with paths) destroyed that shortcut.

Q8. What is the occlusion explanation?

For THIS prediction: replace each feature with a benign-typical value one at a time, re-predict, and report the probability change. Positive = pushed toward phishing. Model-agnostic and intuitive; caveat: ignores feature interactions.

Q9. Why 18 hand-made features instead of the raw string?

Interpretability and parity: every feature has a name, a test, and an explanation, and the same extractor runs in Python and JavaScript. Character-level models could score higher but explain less.

Q10. What does --threshold actually trade off?

Recall vs precision. Threshold 0.3 catches more phishing at the cost of false alarms; 0.9 makes near-certain calls only. Deployment-specific: email filters favor recall, banking sites favor precision.

Q11. Biggest limitation?

Lexical-only vision: we never see page content. Clean-domain-evil-content and homoglyph attacks slip through; the fix is pairing with content/reputation signals in production.

Q12. How would you improve it next?

Scheduled retraining from fresh PhishTank pulls; host-reputation features; character n-gram or transformer URL models for the hard cases; calibration so probabilities mean frequencies.

11

Tooling - What to Install and Why

JupyterLab: your lab notebook for the report

The report needs EDA screenshots and honest experimentation. JupyterLab inside the project venv is perfect for that: load `custom_dataset.csv`, plot class balance, test `extract_features` on your own URLs, and screenshot the outputs for the report's data section.

TERMINAL

```
# fish shell - inside the project folder
.venv/bin/pip install jupyterlab ipykernel

# register THIS venv as a named kernel (so notebooks use project deps)
.venv/bin/python -m ipykernel install --user \
    --name phishing-detector --display-name "Python (phishing-detector)"

# launch
.venv/bin/jupyter lab
```

- Notebook first cell: `import pandas, load data/processed/custom_dataset.csv, df.label.value_counts()` - screenshot that for the report.
- Keep canonical logic in `src/` - notebooks are for exploration and evidence, not for the model code itself.

The rest of the submission toolbox

Tool	For	Get it
LibreOffice Writer / MS Word	The 5,000-word report (Times New Roman 12, 1.5 spacing)	libreoffice.org or preinstalled
LibreOffice Impress / PPT	The presentation deck	Same suite as above
OBS Studio	The 5-6 minute demo video (screen + mic)	obsproject.com or flatpak
Zotero	Harvard-style references, in-text citations, bibliography	zotero.org
Peek / KOHA screenshots	Screenshots of CLI output for the appendix	Built-in screenshot tool is fine

PRO TIP

Record the terminal demo with the font enlarged (one URL per command, ~8 s each). Cut with any editor, add 30 s of slides for architecture and metrics, and your 5-6 minute video is done - script it from the Demo Recipe box in Chapter 6.

Reproduce Everything From Scratch

FULL PIPELINE

```
# 0. environment (fish shell)
cd ~/Documents/AI/phishing-detector
source .venv/bin/activate.fish

# 1. data - raw caches live in data/raw/; --custom rebuilds the dataset
python -m src.fetch_data --custom          # -> data/processed/custom_dataset.csv

# 2. train (also refreshes the web export automatically)
python -m src.train                        # -> models/*.pkl + reports/

# 3. evaluate - metrics.json, confusion_matrix.png, roc_curve.png
python -m src.evaluate

# 4. test the features
python src/features_test.py                # 81/81

# 5. use it
python src/check_url.py https://www.google.com
python src/check_url.py http://paypal.com.verify-login.ru --json

# 6. web demo (optional)
cd ../workspace && npm install && npm run dev  # http://localhost:3000
```

Where the evidence lives

Artifact	Meaning
reports/metrics.json + training_report.json	The real test-set numbers; single source of truth
reports/confusion_matrix.png / roc_curve.png	Visuals to paste into the report
reports/evaluation_summary.md	Written analysis incl. ethics discussion
models/RandomForest.pkl (53 MB, compressed)	200 trees; identical predictions, 5.2x smaller file
data/raw/ (gitignored)	Regenerable caches - the code rebuilds them

Final thought: the model is a mirror of its data. We spent more effort policing the data and the pipeline than tuning the model - and that is exactly why the numbers can be trusted.